SOLID #4 – I: Interface Segregation Principle (ISP)

Plano e documentação base para o quarto repositório, que ilustra o **Princípio da Segregação de Interfaces**.

1. Objetivo do repositório

Mostrar que **nenhum cliente deve ser forçado a depender de métodos que não usa**. Em vez de uma interface grande e genérica, o sistema oferece interfaces pequenas, desenhadas sob medida para quem as consome. O repositório implementa o mesmo sistema em duas versões:

- **Antes:** uma interface "gorda" (`PedidoRepository` com leitura, escrita, exclusão e exportação) usada por todos, obrigando implementações e clientes a lidar com métodos irrelevantes.
- **Depois:** interfaces pequenas e coesas, e cada cliente depende apenas do que realmente utiliza.

O ganho a evidenciar: implementações mais simples, fakes de teste mínimos, menor acoplamento e menos impacto quando uma parte da interface muda.

2. O que é o ISP

> "Os clientes não devem ser forçados a depender de interfaces que não utilizam." — Robert C. Martin

O princípio nasceu de um problema real na Xerox, em que uma classe de impressora usada por vários tipos de tarefa tinha uma interface enorme: qualquer mudança em um método forçava a recompilação e o reteste de clientes que nem usavam aquele método.

Ideia central

A interface deve ser definida **pelo ponto de vista do cliente** ("o que eu preciso?"), e não pelo ponto de vista da implementação ("o que eu consigo fazer?"). Esse estilo é chamado de **role interface** (interface de papel).

Pontos que costumam gerar confusão

- **ISP não é "uma interface com um método".** O objetivo é coesão em relação ao cliente, não o menor tamanho possível.

- **ISP ≠ SRP.** O SRP trata de **motivos de mudança de uma classe**; o ISP trata de **o que o cliente é obrigado a conhecer**. Podem apontar para a mesma solução, mas a pergunta é diferente.
- **Uma classe pode implementar várias interfaces pequenas.** Segregar a interface não obriga a segregar a classe.
- **Relação com o LSP:** uma interface gorda força implementações a lançar `NotImplemented`, que é exatamente a violação vista no repositório anterior. O ISP é a cura estrutural.
- **Relação com o DIP:** as portas que o núcleo define devem ser pequenas e feitas sob medida para o consumidor.

3. Como identificar uma violação (code smells)

- Interfaces com muitos métodos (por exemplo, mais de cinco ou seis) que misturam assuntos diferentes.
- Implementações com métodos vazios, `return null` ou `throw new NotImplementedException()`.
- Clientes que usam apenas um ou dois métodos de uma interface grande.
- Fakes e mocks de teste enormes, com muitos métodos irrelevantes para o cenário testado.
- Mudança em um método da interface obriga a alterar ou recompilar classes que nunca o chamaram.
- Nomes genéricos como `IService`, `IManager`, `IRepository` com tudo dentro.
- Comentários do tipo "não use este método se a implementação for X".

4. Cenário proposto: repositório de pedidos com muitos clientes

Reaproveitar o domínio da série (**Pedido, Cliente, Item**) e olhar para o repositório de pedidos, que costuma virar uma interface gorda. Cada cliente precisa de uma parte diferente:

Cliente (consumidor)	O que realmente precisa
`FinalizarPedido`	Salvar
`ConsultarPedido` (tela de acompanhamento)	Buscar por id, listar
`CancelarPedido`	Buscar por id, excluir
`RelatorioFinanceiro`	Listar (somente leitura)

Cliente (consumidor)	O que realmente precisa
Réplica de leitura / cache	Apenas leitura (não escreve)

4.1 Versão que VIOLA o ISP

```
interface PedidoRepository {
  salvar(pedido: Pedido): void;
  buscarPorId(id: string): Pedido | null;
  listar(): Pedido[];
  excluir(id: string): void;
  exportarCsv(): string;
}

// Uma réplica de leitura não pode escrever, mas é obrigada a implementar
tudo
class PedidoRepositoryReplica implements PedidoRepository {
  salvar(pedido: Pedido) { throw new Error("Réplica é somente leitura"); }
  buscarPorId(id: string) { /* ... */ }
  listar() { /* ... */ }
  excluir(id: string) { throw new Error("Réplica é somente leitura"); }
  exportarCsv() { /* ... */ }
}

class FinalizarPedido {
  constructor(private repo: PedidoRepository) {}   // só usa salvar(), mas
enxerga tudo
  executar(p: Pedido) { this.repo.salvar(p); }
}

class RelatorioFinanceiro {
  constructor(private repo: PedidoRepository) {}   // só lê, mas poderia
excluir
  gerar() { return this.repo.listar(); }
}
```

Por que é um problema?

- A réplica é obrigada a implementar o que não pode fazer e acaba lançando exceção (violação do LSP).
- `RelatorioFinanceiro` tem acesso a `excluir`, embora só precise ler: uma dependência desnecessária e arriscada.
- Para testar `FinalizarPedido`, o fake precisa implementar cinco métodos, sendo que só um importa.

- Se `exportarCsv` mudar de assinatura, todos os clientes e implementações são afetados, inclusive os que nunca o usam.

4.2 Versão que RESPEITA o ISP

```
interface SalvaPedido    { salvar(pedido: Pedido): void; }
interface BuscaPedido    { buscarPorId(id: string): Pedido | null; }
interface ListaPedidos   { listar(): Pedido[]; }
interface ExcluiPedido   { excluir(id: string): void; }
interface ExportaPedidos { exportarCsv(): string; }

// Uma implementação completa pode implementar várias interfaces
class MySqlPedidoRepository
  implements SalvaPedido, BuscaPedido, ListaPedidos, ExcluiPedido,
ExportaPedidos { /* ... */ }

// A réplica implementa só o que realmente faz
class PedidoRepositoryReplica implements BuscaPedido, ListaPedidos { /* ...
*/ }

// Cada cliente depende apenas do que usa
class FinalizarPedido {
  constructor(private repo: SalvaPedido) {}
}

class CancelarPedido {
  constructor(private busca: BuscaPedido, private exclui: ExcluiPedido) {}
}

class RelatorioFinanceiro {
  constructor(private pedidos: ListaPedidos) {}
}
```

Agora a réplica não precisa fingir que escreve, o relatório não enxerga `excluir`, e o teste de `FinalizarPedido` usa um fake de **uma linha**.

> Os exemplos estão em TypeScript apenas como referência; adapte para a linguagem escolhida.

4.3 Segunda demonstração: o fake de teste encolhe

```
// Antes: o fake precisa de cinco métodos para testar um
class RepoFalso implements PedidoRepository {
  salvar(p: Pedido) { /* ... */ }
  buscarPorId() { return null; }
  listar() { return []; }
  excluir() {}
  exportarCsv() { return ""; }
}

// Depois: o fake tem exatamente o que o cliente precisa
const repoFalso: SalvaPedido = { salvar: (p) => salvos.push(p) };
```

4.4 Exemplo clássico complementar (opcional): impressora multifuncional

Um exemplo curto e didático para reforçar a ideia: uma interface `MaquinaMultifuncional` com `imprimir`, `escanear` e `enviarFax`, que força uma `ImpressoraSimples` a implementar métodos que ela não suporta. A correção é separar em `Impressora`, `Scanner` e `Fax`, e compor na `Multifuncional`.

5. Técnicas para aplicar o ISP

Técnica	Quando usar	Exemplo
Role interfaces	Cada cliente tem uma necessidade distinta	`SalvaPedido`, `ListaPedidos`
Extrair interface (Extract Interface)	Uma interface grande já existe e precisa ser dividida	Dividir `PedidoRepository` em papéis
Composição de interfaces	Um cliente precisa de mais de um papel	`BuscaPedido` e `ExcluiPedido` juntos
Separar leitura e escrita (CQS/CQRS leve)	Leitura e escrita têm necessidades diferentes	`Reader` e `Writer`
Adapter	Um código legado expõe a interface gorda e não pode mudar	Adaptador que expõe só o papel necessário

O repositório deve focar em **role interfaces + extração de interface**; a separação leitura/escrita pode ser citada como extra.

6. Estrutura sugerida do repositório

```
solid-04-isp/
├── README.md
├── docs/
│   └── isp.md                # este documento
├── before/                   # interface gorda
│   ├── src/
│   └── tests/
├── after/                    # interfaces segregadas
│   ├── src/
│   │   ├── domain/           # Pedido, Item, Cliente
│   │   ├── application/      # casos de uso + role interfaces (ports)
│   │   └── infra/            # MySql..., Replica..., InMemory...
│   └── tests/
│       └── unit/             # fakes mínimos, um por papel
└── CHANGELOG.md
```

7. Roteiro de implementação (plano de trabalho)

1. **Base:** copiar o domínio e os casos de uso da série (Pedido, Cliente, Item, `FinalizarPedido`, `CancelarPedido`).
2. **Versão `before`:** criar `PedidoRepository` com cinco métodos, a implementação completa e a réplica de leitura.
3. **Demonstrar a dor:** registrar no `CHANGELOG` os métodos que a réplica foi obrigada a implementar com exceção e o tamanho do fake necessário para testar `FinalizarPedido`.
4. **Mapear os clientes:** montar uma tabela com o que cada caso de uso **realmente** chama.
5. **Segregar:** extrair as role interfaces a partir dessa tabela, e não pelo que a implementação consegue fazer.
6. **Ajustar as implementações:** a classe completa implementa todos os papéis; a réplica só os de leitura.
7. **Ajustar os clientes:** cada caso de uso passa a receber apenas os papéis de que precisa.
8. **Testes:** unitários com fakes de um método; teste que prove que a réplica não pode ser passada onde se exige escrita (erro de compilação ou tipo).
9. **Provar a evolução:** adicionar um novo método ao papel `ExportaPedidos` e registrar que **nenhum** cliente fora dele precisou mudar.
10. **Documentar:** README com o comparativo antes vs. depois e como rodar.

8. Requisitos de implementação

- Mesmo comportamento externo nas duas versões (os testes de aceitação passam em ambas).
- Em `after`, nenhuma implementação pode ter método vazio, `throw NotImplemented` ou `return null` só para cumprir a interface.
- Cada cliente declara em seu construtor apenas as interfaces que usa.
- Interfaces nomeadas pelo **papel** (`SalvaPedido`), e não pelo tipo de tecnologia (`IMySql`).
- Fakes de teste com o mínimo de métodos possível, escritos à mão.
- As interfaces pertencem à camada de alto nível, mantendo a regra do DIP.
- Sem frameworks desnecessários; o foco é o princípio.

9. Métricas para provar o ganho

Métrica	Antes	Depois
Métodos da interface usados por `FinalizarPedido`	1 de 5	1 de 1
Implementações com método que lança `NotImplemented`	1 ou mais	0
Métodos no fake de teste de `FinalizarPedido`	5	1
Clientes afetados ao mudar `exportarCsv`	Todos	Apenas quem usa `ExportaPedidos`

10. Cuidados: o outro extremo

- **Pulverização:** uma interface por método, em todo lugar, gera dezenas de tipos e construtores com muitos parâmetros. Agrupe métodos que **sempre** são usados juntos pelos mesmos clientes (por exemplo, `buscarPorId` e `listar` em um papel `ConsultaPedidos`, se todos os consumidores precisarem dos dois).
- **Construtor com muitas interfaces:** se um cliente precisa de cinco papéis, talvez ele tenha responsabilidades demais (volte ao SRP).
- **Não segregue por antecipação.** Divida quando houver clientes reais com necessidades diferentes, e não "por via das dúvidas".

- **Interface por implementação:** desenhar a interface a partir do que a classe faz (e não do que o cliente precisa) recria a interface gorda.
- **Atenção ao legado:** em código existente, extraia os papéis aos poucos, usando *Adapter* ou deixando a interface antiga herdar das novas durante a transição.

11. Checklist de revisão

- [ ] Cada cliente depende apenas dos métodos que realmente usa?
- [ ] Nenhuma implementação lança `NotImplemented`, tem método vazio ou retorna valor falso para cumprir a interface?
- [ ] As interfaces foram definidas a partir das necessidades dos clientes?
- [ ] Os nomes das interfaces revelam o papel?
- [ ] Os fakes de teste são mínimos?
- [ ] Mudar um método afeta somente os clientes que o usam?
- [ ] Não houve pulverização excessiva (interfaces de um método sem necessidade)?
- [ ] O README explica o problema, a solução e como rodar?

12. Modelo de README do repositório

1. **Título e objetivo** — qual princípio é ilustrado.
2. **Conceito** — definição do ISP em poucas linhas, com a ideia de *role interface*.
3. **Problema** — a dor de `before` (a interface gorda e a réplica forçada a lançar exceção).
4. **Solução** — como `after` resolve (diagrama com os papéis e quem os consome).
5. **Como executar** — instalação, build e testes.
6. **Comparativo** — tabela de métricas.
7. **Referências.**

13. Referências

- Robert C. Martin — "The Interface Segregation Principle" (C++ Report, 1996).
- Robert C. Martin — *Agile Software Development: Principles, Patterns, and Practices* e *Clean Architecture*.
- Martin Fowler — "RoleInterface" (bliki) e a refatoração *Extract Interface*.
- Gang of Four — *Design Patterns* (Adapter).

14. Próximo passo

Com os planos de S, O, L, I e D prontos, os cinco repositórios já compartilham o mesmo domínio de pedidos. O passo natural é implementar cada um na ordem `solid-01-srp` a `solid-05-dip` e, ao final, criar um repositório integrado (`solid-all`) em que o sistema aplica os cinco princípios juntos.